- You can chain `styled` with an HTML tag name, as we did in this code with `span`. In this case, the result is a React component where that HTML tag is the top-level tag of the result.

- You can use the name of another React component as an argument to `styled`. We can even use this to extend our existing component, such as `styled(Header)`.

In either case we end up with a function that directly takes our backtick string (technically called a *tagged template literal*) as an argument. There's some internal JavaScript mechanics going on here, which is why the code is `styled.span‘css‘` and not `styled.span(‘css‘)`, but the details aren't important. What is important is that it's a JavaScript backtick literal, which means we can use `${}` interpolations inside the string. (Although it's not the preferred method, you can pass a JavaScript object with key/values rather than a backtick string.)

The resulting value is a regular React component that can be used like any other React component.

Dynamic styled-components

Next, let's add some additional styling to our actual `Seat` component. First, though, I see two concerns: our existing `Seat` uses a lot of Tailwind classes for styling, and we'd like to keep them. Second, we've been adjusting the background color of the seats based on their status, and we need to continue to do that. The styled-components library allows us to do both of these things.

The key point here is that the components that are returned using `styled-components` are regular React components, which means they can be passed props and you can use those props inside the literal template string to dynamically change the value.

Here's what the `Seat` looks like with some refactoring to take advantage of styled-components:

chapter_05/07/app/javascript/components/seat.tsx